

Database Basics

```
-- Just creating a simple database to store employee data
CREATE DATABASE EmployeeDB;

-- Cleaning up by deleting a sample test database
DROP DATABASE IF EXISTS TestDB;
```

Database Selection

```
-- Switching to the EmployeeDB to start working with it
USE EmployeeDB;

-- Just confirming we're in the right place
SELECT DATABASE();
```

Schema Design

```
-- Planning the Employees table with basic fields
CREATE TABLE Employees (
    ID INT PRIMARY KEY,
    Name VARCHAR(100),
    JoinDate DATE
);

-- Creating another table for departments to show relationships later
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(100)
);
```

Creating Tables

```
-- Creating a Customers table for an imaginary store
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    FullName VARCHAR(100),
    Email VARCHAR(255)
);

-- Making a simple Products table with price info
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(100),
    Price DECIMAL(10, 2)
);
```

Modifying Tables (ALTER TABLE)

```
-- Adding a new column to store phone number
ALTER TABLE Customers ADD Phone VARCHAR(15);

-- Changing the column name to make it more descriptive
ALTER TABLE Products RENAME COLUMN Price TO UnitPrice;
```

Dropping Tables

```
-- Dropping an unused table from an old project
DROP TABLE IF EXISTS Old_Orders;

-- Removing a test table we don't need anymore
DROP TABLE IF EXISTS Temp_Users;
```

Truncating Tables

```
-- Clearing all records from Products without deleting the table
TRUNCATE TABLE Products;

-- Resetting Customers table to start fresh
TRUNCATE TABLE Customers;
```

Data Types

Numeric (INT, DECIMAL, FLOAT)

```
-- Using DECIMAL for accurate monetary values
CREATE TABLE Prices (
    ItemID INT,
    Cost DECIMAL(10, 2)
);

-- Using FLOAT where exact precision isn't critical
CREATE TABLE Measurements (
    MeasureID INT,
    Value FLOAT
);
```

String (CHAR, VARCHAR, TEXT)

```
-- CHAR for fixed-length strings like codes
CREATE TABLE CountryCodes (
    Code CHAR(3),
    CountryName VARCHAR(100)
);

-- TEXT for long descriptions
```

```
CREATE TABLE Articles (  
    Title VARCHAR(100),  
    Body TEXT  
);
```

Date and Time (DATE, DATETIME, TIMESTAMP)

```
-- DATE to store just the birthday  
CREATE TABLE Birthdays (  
    PersonName VARCHAR(100),  
    BirthDate DATE  
);
```

```
-- DATETIME to record logins  
CREATE TABLE LoginRecords (  
    UserID INT,  
    LoginTime DATETIME  
);
```

Data Manipulation Language (DML)

INSERT

```
-- Adding a new customer  
INSERT INTO Customers (CustomerID, FullName, Email) VALUES (1, 'Alice Smith',  
'alice@example.com');  
  
-- Inserting a new product  
INSERT INTO Products (ProductID, ProductName, UnitPrice) VALUES (1,  
'Notebook', 49.99);
```

SELECT

```
-- Getting all customers  
SELECT * FROM Customers;  
  
-- Viewing product names and prices only  
SELECT ProductName, UnitPrice FROM Products;
```

UPDATE

```
-- Changing a customer's email  
UPDATE Customers SET Email = 'newalice@example.com' WHERE CustomerID = 1;  
  
-- Reducing the price of a product  
UPDATE Products SET UnitPrice = UnitPrice - 5 WHERE ProductID = 1;
```

DELETE

```
-- Deleting a customer who left  
DELETE FROM Customers WHERE CustomerID = 1;  
  
-- Removing all products above a certain price
```

```
DELETE FROM Products WHERE UnitPrice > 100;
```

Data Definition Language (DDL)

CREATE

```
-- Creating a table for orders
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE
);

-- Making a payments table
CREATE TABLE Payments (
    PaymentID INT,
    Amount DECIMAL(10, 2)
);
```

ALTER

```
-- Adding status to orders
ALTER TABLE Orders ADD Status VARCHAR(20);

-- Modifying amount column size
ALTER TABLE Payments MODIFY Amount DECIMAL(12, 2);
```

DROP

```
-- Removing the Payments table
DROP TABLE Payments;

-- Dropping Orders table to rebuild from scratch
DROP TABLE Orders;
```

TRUNCATE

```
-- Cleaning all orders data before a new sales season
TRUNCATE TABLE Orders;

-- Resetting a log table
TRUNCATE TABLE LoginRecords;
```

Constraints

PRIMARY KEY

```
-- Setting ID as primary key
CREATE TABLE Employees (
    ID INT PRIMARY KEY,
    Name VARCHAR(100)
);

-- Another primary key example for departments
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(100)
);
```

FOREIGN KEY

```
-- Linking Employees to Departments
ALTER TABLE Employees ADD DepartmentID INT;
ALTER TABLE Employees
ADD FOREIGN KEY (DepartmentID) REFERENCES Departments(DeptID);

-- Foreign key in orders
ALTER TABLE Orders
ADD FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID);
```

UNIQUE

```
-- Making sure emails are unique
ALTER TABLE Customers ADD CONSTRAINT UNIQUE (Email);

-- Unique username in a users table
CREATE TABLE Users (
    ID INT,
    Username VARCHAR(50) UNIQUE
);
```

NOT NULL

```
-- Name should never be null
ALTER TABLE Employees MODIFY Name VARCHAR(100) NOT NULL;

-- Ensuring OrderDate is always given
ALTER TABLE Orders MODIFY OrderDate DATE NOT NULL;
```

CHECK

```
-- Age must be above 18
CREATE TABLE People (
    ID INT,
    Age INT CHECK (Age >= 18)
);
```

```
-- Price should be positive
ALTER TABLE Products ADD CHECK (UnitPrice > 0);
```

DEFAULT

```
-- Default country is set to Bangladesh
ALTER TABLE Customers ADD Country VARCHAR(50) DEFAULT 'Bangladesh';

-- Status default is 'Pending'
ALTER TABLE Orders MODIFY Status VARCHAR(20) DEFAULT 'Pending';
```

Indexes

Creating Indexes

```
-- Index to speed up name searches
CREATE INDEX idx_name ON Employees(Name);

-- Index for quick email lookup
CREATE INDEX idx_email ON Customers(Email);
```

Clustered vs Non-Clustered Indexes

(Note: MySQL uses clustered indexes by default on PRIMARY KEYs)

```
-- Implicit clustered index on primary key
CREATE TABLE Books (
    BookID INT PRIMARY KEY,
    Title VARCHAR(100)
);

-- Explicit non-clustered index (behavior depends on DBMS)
CREATE INDEX idx_title ON Books(Title);
```

Unique Indexes

```
-- Enforcing uniqueness through index
CREATE UNIQUE INDEX idx_unique_email ON Customers(Email);

-- Unique username index
CREATE UNIQUE INDEX idx_username ON Users(Username);
```

Composite Indexes

```
-- Index combining first and last name
```

```
CREATE INDEX idx_fullname ON Employees (Name, ID);

-- Index for product name and price
CREATE INDEX idx_product_price ON Products (ProductName, UnitPrice);
```

Dropping Indexes

```
-- Removing name index
DROP INDEX idx_name ON Employees;

-- Dropping unique email index
DROP INDEX idx_unique_email ON Customers;
```

Queries and Joins

Basic SELECT Queries

```
-- View all employees
SELECT * FROM Employees;

-- See names and join dates only
SELECT Name, JoinDate FROM Employees;
```

WHERE Clause

```
-- Find employees who joined after 2020
SELECT * FROM Employees WHERE JoinDate > '2020-01-01';

-- Get expensive products
SELECT * FROM Products WHERE UnitPrice > 100;
```

ORDER BY

```
-- List products by price, high to low
SELECT * FROM Products ORDER BY UnitPrice DESC;

-- Order employees alphabetically
SELECT * FROM Employees ORDER BY Name ASC;
```

GROUP BY

```
-- Count how many employees in each department
SELECT DepartmentID, COUNT(*) FROM Employees GROUP BY DepartmentID;

-- Total sales per customer
SELECT CustomerID, SUM(UnitPrice) FROM Orders GROUP BY CustomerID;
```

HAVING

```
-- Departments with more than 5 employees
```

```
SELECT DepartmentID, COUNT(*) AS Total FROM Employees GROUP BY DepartmentID
HAVING Total > 5;
```

```
-- Customers who bought over 500 worth
SELECT CustomerID, SUM(UnitPrice) AS Total FROM Orders GROUP BY CustomerID
HAVING Total > 500;
```

Subquery with SELECT Statement

Use case: Retrieve data based on a condition derived from another query.

Example 1: Get names of employees who earn more than the average salary

```
SELECT name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Example 2: Get names of products priced higher than the average product price

```
SELECT name
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Subquery with INSERT Statement

Use case: Insert data retrieved from another query.

Example 1: Insert all top-performing students into the honors table

```
INSERT INTO honors_students (student_id, name)
SELECT id, name
FROM students
WHERE gpa > (SELECT AVG(gpa) FROM students);
```

Example 2: Insert new customers from leads table where interest is high

```
INSERT INTO customers (name, email)
SELECT name, email
FROM leads
WHERE interest = 'High';
```

Subquery with UPDATE Statement

Use case: Update values in a table using data from another table/query.

Example 1: Give a 10% salary hike to employees who work in the 'R&D' department

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id = (
```



```
    SELECT id FROM departments WHERE name = 'R&D'
);
```

Example 2: Update product price by 5% for products in category 'Electronics'

```
UPDATE products
SET price = price * 1.05
WHERE category_id = (
    SELECT id FROM categories WHERE name = 'Electronics'
);
```

Subquery with DELETE Statement

Use case: Delete records that match a condition fetched from another query.

Example 1: Delete orders placed by inactive customers

```
DELETE FROM orders
WHERE customer_id IN (
    SELECT id FROM customers WHERE status = 'inactive'
);
```

Example 2: Delete reviews related to discontinued products

```
DELETE FROM reviews
WHERE product_id IN (
    SELECT id FROM products WHERE status = 'discontinued'
);
```

Subquery inside WHERE Clause

Use case: Use subquery to filter rows with a condition.

Example 1: Get names of students enrolled in course 'Database Systems'

```
SELECT name
FROM students
WHERE id IN (
    SELECT student_id
    FROM enrollments
    WHERE course_id = (
        SELECT id FROM courses WHERE name = 'Database Systems'
    )
);
```

Example 2: Get names of employees in the same department as 'John'

```
SELECT name
FROM employees
WHERE department_id = (
    SELECT department_id FROM employees WHERE name = 'John'
);
```

```
);
```

Subquery with Operators

= Operator

Example 1: Get employees in the 'HR' department

```
SELECT name FROM employees
WHERE department_id = (SELECT id FROM departments WHERE name = 'HR');
```

Example 2: Get students in the 'Physics' department

```
SELECT name FROM students
WHERE department_id = (SELECT id FROM departments WHERE name = 'Physics');
```

< or > Operator

Example 1: Get products priced below average

```
SELECT name FROM products
WHERE price < (SELECT AVG(price) FROM products);
```

Example 2: Get employees older than the average employee age

```
SELECT name FROM employees
WHERE age > (SELECT AVG(age) FROM employees);
```

IN Operator

Example 1: Get names of students in the honors list

```
SELECT name FROM students
WHERE id IN (
    SELECT student_id FROM honors_list
);
```

Example 2: Get products ordered by customer with ID 101

```
SELECT name FROM products
WHERE id IN (
    SELECT product_id FROM orders WHERE customer_id = 101
);
```

BETWEEN Operator

Example 1: Get students with GPA in the 'B' grade range

```
SELECT name FROM students
WHERE gpa BETWEEN (
    SELECT MIN(gpa) FROM students WHERE grade = 'B'
) AND (
    SELECT MAX(gpa) FROM students WHERE grade = 'B'
);
```

```
);
```

Example 2: Get employees with salaries in the mid-range

```
SELECT name FROM employees
WHERE salary BETWEEN (
    SELECT MIN(salary) FROM employees WHERE position = 'Senior'
) AND (
    SELECT MAX(salary) FROM employees WHERE position = 'Junior'
);
```

INNER JOIN

```
-- Join employees with departments
SELECT E.Name, D.DeptName
FROM Employees E
INNER JOIN Departments D ON E.DepartmentID = D.DeptID;

-- Products with their orders
SELECT O.OrderID, P.ProductName
FROM Orders O
INNER JOIN Products P ON O.ProductID = P.ProductID;
```

LEFT JOIN

```
-- Show all employees, even if not assigned a department
SELECT E.Name, D.DeptName
FROM Employees E
LEFT JOIN Departments D ON E.DepartmentID = D.DeptID;

-- Customers and their orders (including those with no orders)
SELECT C.FullName, O.OrderID
FROM Customers C
LEFT JOIN Orders O ON C.CustomerID = O.CustomerID;
```

RIGHT JOIN

```
-- All departments, even without employees
SELECT E.Name, D.DeptName
FROM Employees E
RIGHT JOIN Departments D ON E.DepartmentID = D.DeptID;

-- Products and orders (include products not ordered)
SELECT O.OrderID, P.ProductName
FROM Orders O
RIGHT JOIN Products P ON O.ProductID = P.ProductID;
```

At a Glance:

1. `SELECT * FROM INFO;`
→ Retrieves all rows and columns from the INFO table.
2. `SET SQL_SAFE_UPDATES=0;`
→ Turns off safe update mode to allow updates without key conditions (MySQL specific).
3. `ALTER TABLE INFO ADD COLUMN NON INT;`
→ Adds a new integer column named NON.
4. `ALTER TABLE INFO DROP COLUMN NON;`
→ Deletes the column named NON.
5. `UPDATE INFO SET CITY="DHAKA" WHERE NAME="GURUUU";`
→ Sets CITY to "DHAKA" for NAME "GURUUU".
6. `SELECT NAME FROM INFO;`
→ Shows only the NAME column from INFO.
7. `SELECT DISTINCT CITY FROM INFO;`
→ Returns unique CITY names.
8. `SELECT NAME FROM INFO WHERE AGE>=23;`
→ Finds names with AGE 23 or above.
9. `SELECT NAME FROM INFO WHERE AGE!=23;`
→ Finds names where AGE is not 23.
10. `SELECT NAME FROM INFO WHERE CITY="DHAKA" or CITY="CUMILLA";`
→ valid
11. `SELECT NAME FROM INFO WHERE CITY="DHAKA" OR CITY="CUMILLA";`
→ Finds names from DHAKA or CUMILLA.
12. `SELECT NAME FROM INFO WHERE NOT AGE=23;`
→ Same as AGE != 23.
13. `SELECT ID,NAME FROM INFO WHERE (CITY="DHAKA" OR CITY="CUMILLA") AND AGE=23;`
→ Gets ID and NAME of people aged 23 from DHAKA or CUMILLA.
14. `SELECT * FROM INFO WHERE CITY IN("DHAKA", "CUMILLA");`
→ Same as using OR for multiple cities.
15. `SELECT * FROM INFO WHERE AGE BETWEEN 20 AND 25;`
→ Gets records with AGE between 20 and 25 inclusive.
16. `SELECT * FROM INFO WHERE NAME LIKE "M%";`
→ Names starting with M.
17. `SELECT * FROM INFO WHERE NAME LIKE "%A%";`
→ Names containing A.
18. `SELECT * FROM INFO WHERE NAME LIKE "%A";`
→ Names ending with A.
19. `SELECT * FROM INFO WHERE NAME LIKE "A%A"`
→ Names starting and ending with A.
20. `SELECT * FROM INFO WHERE NAME="NISHAT";`
→ Gets data of person named NISHAT.

21. `SELECT * FROM INFO WHERE NAME="ABIR";`
→ Gets data of person named ABIR.
22. `SELECT MIN(AGE) FROM INFO;`
→ Returns the lowest age.
23. `SELECT MAX(AGE) FROM INFO;`
→ Returns the highest age.
24. `SELECT COUNT(AGE) FROM INFO;`
→ Counts how many non-null AGE values exist.
25. `SELECT AVG(AGE) FROM INFO;`
→ Calculates average age.
26. `SELECT SUM(AGE) FROM INFO;`
→ Adds all AGE values.
27. `UPDATE INFO SET AGE=AGE+3 WHERE ID=101;`
→ Increases AGE by 3 for ID 101.
28. `UPDATE INFO SET AGE=AGE-3 WHERE ID=101;`
→ Decreases AGE by 3 for ID 101.
29. `SELECT SUM(AGE) AS TOTAL_AGE FROM INFO WHERE NAME IN ("ABIR","NISHAT")`
`GROUP BY NAME;`
→ Sums age separately for ABIR and NISHAT.
30. `SELECT SUM(AGE) AS TOTAL_AGE FROM INFO WHERE NAME IN ("ABIR",`
`"NISHAT");`
→ Sums total age of ABIR and NISHAT combined